Machine Learning (Classical) - Supervised, Unsupervised, Evaluation

All core ML algorithms with intuition, formula idea, scikit-learn code and when to use them.

Contents

#	Topic	Level
1	What is ML? Types and the ML workflow	Beginner
2	Bias-variance trade-off, overfitting and underfitting	Beginner
3	Linear Regression	Beginner
4	Polynomial Regression	Beginner
5	Regularisation: Ridge (L2), Lasso (L1), ElasticNet	Intermediate
6	Logistic Regression	Beginner
7	K-Nearest Neighbours (KNN)	Beginner
8	Naive Bayes	Beginner
9	Decision Trees	Beginner
10	Ensemble learning: Bagging and Random Forest	Intermediate
11	Boosting: AdaBoost, Gradient Boosting, XGBoost, LightGBM, CatBoost	Intermediate
12	Stacking and Voting ensembles	Advanced
13	Support Vector Machines (SVM) and kernels	Intermediate
14	Model evaluation: metrics for classification	Beginner
15	Model evaluation: metrics for regression	Beginner
16	Cross-validation, grid/random search, hyper-parameter tuning	Intermediate
17	Probability calibration and threshold selection	Advanced
18	Clustering: K-Means, K-Means++, elbow and silhouette	Beginner
19	Clustering: Hierarchical, DBSCAN, Gaussian Mixture, Spectral	Intermediate
20	Anomaly / outlier detection	Intermediate
21	Association rules: Apriori / FP-Growth (market basket)	Intermediate
22	Recommender systems: content-based and collaborative filtering	Intermediate
23	Semi-supervised, self-supervised and active learning	Advanced
24	Time-series forecasting (classical): moving average, ARIMA, SARIMA, Prophet	Intermediate
25	Interpretability: feature importance, permutation importance, PDP, SHAP, LIME	Advanced
26	Saving models and reproducibility	Beginner
27	End-to-end ML project template	Intermediate

1. What is ML? Types and the ML workflow [Beginner]

Explanation (English)

ML lets computers learn patterns from data. Types: supervised (labelled), unsupervised (unlabelled), semi/self-supervised, reinforcement. Workflow: define problem -> collect data -> clean -> features -> train -> evaluate -> deploy -> monitor.

Tanglish (Tamil + English) Vilakkam

ML = data-la irundhu pattern kathukkradhu. Supervised, unsupervised, reinforcement. Workflow: problem, data, clean, feature, train, evaluate, deploy.

Learn more

YouTube search: what is machine learning types supervised unsupervised

2. Bias-variance trade-off, overfitting and underfitting [Beginner]

Explanation (English)

Underfitting = high bias (too simple); overfitting = high variance (memorises noise). Fix with more data, regularisation, simpler/complex models, early stopping, cross-validation.

Tanglish (Tamil + English) Vilakkam

Underfit = model romba simple. Overfit = training-la nalla, test-la mosam. Regularization, more data, CV.

Example / Program

```
# compare train vs test score
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
X,y=load_iris(return_X_y=True)
Xtr,Xte,ytr,yte=train_test_split(X,y,random_state=0)
m=DecisionTreeClassifier().fit(Xtr,ytr);print(m.score(Xtr,ytr),m.score(Xte,yte))
```

Learn more

YouTube search: Bias-variance trade-off, overfitting and underfitting tutorial

3. Linear Regression [Beginner]

Explanation (English)

Fits y = w.x + b by minimising squared error (closed form or gradient descent). Assumes linearity, independence, homoscedasticity, normal residuals. Metrics: MAE, MSE, RMSE, R2.

Tanglish (Tamil + English) Vilakkam

Linear regression straight line fit pannum. Error = (y - y_pred)^2 minimize. R2, RMSE metric.

Example / Program

```
from sklearn.linear_model import LinearRegression
from sklearn.datasets import make_regression
X,y=make_regression(100,1,noise=10,random_state=0)
m=LinearRegression().fit(X,y);print(m.coef_,m.intercept_,m.score(X,y))
```

Learn more

YouTube search: Linear Regression tutorial

4. Polynomial Regression [Beginner]

Explanation (English)

Linear regression on polynomial features; models curves but overfits at high degree.

Tanglish (Tamil + English) Vilakkam

Curve data-ku polynomial features + linear regression.

Example / Program

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LinearRegression
m=make_pipeline(PolynomialFeatures(3),LinearRegression())
```

Learn more

YouTube search: Polynomial Regression tutorial

5. Regularisation: Ridge (L2), Lasso (L1), ElasticNet [Intermediate]

Explanation (English)

Penalise large weights: Ridge shrinks them, Lasso drives some to zero (feature selection), ElasticNet mixes both. Strength via alpha.

Tanglish (Tamil + English) Vilakkam

Ridge weights-a shrink pannum, Lasso zero aakkum. alpha tune pannunga.

Example / Program

```
from sklearn.linear_model import Ridge,Lasso,ElasticNet
m=Lasso(alpha=0.1)
```

Learn more

YouTube search: Regularisation: Ridge (L2), Lasso (L1), ElasticNet tutorial

6. Logistic Regression [Beginner]

Explanation (English)

Binary classifier: p = sigmoid(w.x+b), trained with cross-entropy (log loss). Extends to multi-class with softmax. Outputs probabilities; coefficients are interpretable as log-odds.

Tanglish (Tamil + English) Vilakkam

Logistic regression classification-ku. Sigmoid probability tharum. Threshold 0.5.

Example / Program

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
X,y=load_iris(return_X_y=True)
Xtr,Xte,ytr,yte=train_test_split(X,y,random_state=0)
m=LogisticRegression(max_iter=200).fit(Xtr,ytr);print(m.score(Xte,yte),m.predict_proba(Xte[:2]))
```

Learn more

YouTube search: Logistic Regression tutorial

7. K-Nearest Neighbours (KNN) [Beginner]

Explanation (English)

Predict by majority/average of the k closest training points. Lazy learner; needs scaling; slow for large data; curse of dimensionality.

Tanglish (Tamil + English) Vilakkam

KNN: nearest k neighbours vote. Scaling must. Large data-ku slow.

Example / Program

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
X,y=load_iris(return_X_y=True)
Xtr,Xte,ytr,yte=train_test_split(X,y,random_state=0)
print(KNeighborsClassifier(5).fit(Xtr,ytr).score(Xte,yte))
```

Learn more

YouTube search: K-Nearest Neighbours (KNN) tutorial

8. Naive Bayes [Beginner]

Explanation (English)

Applies Bayes' theorem assuming features are conditionally independent. Gaussian, Multinomial, Bernoulli variants. Fast and strong baseline for text/spam.

Tanglish (Tamil + English) Vilakkam

Naive Bayes - features independent-nu assume panni Bayes theorem. Spam filter-ku super.

Example / Program

```
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
X,y=load_iris(return_X_y=True)
```

```
Xtr,Xte,ytr,yte=train_test_split(X,y,random_state=0)
print(GaussianNB().fit(Xtr,ytr).score(Xte,yte))
```

Learn more
YouTube search: Naive Bayes tutorial

9. Decision Trees [Beginner]

Explanation (English)
Split data on features using Gini impurity or entropy / information gain (classification) or variance reduction (regression). Interpretable but prone to overfitting; control with max_depth, min_samples_leaf, pruning.

Tanglish (Tamil + English) Vilakkam
Decision tree if-else rules maadhiri. Gini/entropy-la split. max_depth control pannunga.

Example / Program
```
from sklearn.tree import DecisionTreeClassifier,export_text
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
X,y=load_iris(return_X_y=True)
Xtr,Xte,ytr,yte=train_test_split(X,y,random_state=0)
t=DecisionTreeClassifier(max_depth=3).fit(Xtr,ytr);print(export_text(t))
```

Learn more
YouTube search: Decision Trees tutorial

10. Ensemble learning: Bagging and Random Forest [Intermediate]

Explanation (English)
Bagging trains many models on bootstrap samples and averages them (reduces variance). Random Forest adds random feature subsets; gives feature importance and OOB score.

Tanglish (Tamil + English) Vilakkam
Bagging: pala trees average. Random Forest: robust, kammi tuning.

Example / Program
```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
X,y=load_iris(return_X_y=True)
Xtr,Xte,ytr,yte=train_test_split(X,y,random_state=0)
m=RandomForestClassifier(200,random_state=0).fit(Xtr,ytr);print(m.score(Xte,yte),m.feature_importances_)
```

Learn more
YouTube search: Ensemble learning: Bagging and Random Forest tutorial

11. Boosting: AdaBoost, Gradient Boosting, XGBoost, LightGBM, CatBoost [Intermediate]

Explanation (English)
Boosting trains models sequentially, each correcting previous errors (reduces bias). XGBoost/LightGBM/CatBoost add regularisation, speed, categorical handling. Top choice for tabular data.

Tanglish (Tamil + English) Vilakkam
Boosting: oru model seitha thappa adutha model sari pannum. Tabular data-la best.

Example / Program
```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
X,y=load_iris(return_X_y=True)
Xtr,Xte,ytr,yte=train_test_split(X,y,random_state=0)
print(GradientBoostingClassifier().fit(Xtr,ytr).score(Xte,yte))
```

Learn more
YouTube search: Boosting: AdaBoost, Gradient Boosting, XGBoost, LightGBM, CatBoost tutorial

12. Stacking and Voting ensembles [Advanced]

Explanation (English)
Combine diverse models: voting averages predictions; stacking trains a meta-model on base-model outputs.

Tanglish (Tamil + English) Vilakkam
Pala models-a serthu final prediction - voting, stacking.

Example / Program
```
from sklearn.ensemble import VotingClassifier,RandomForestClassifier
from sklearn.linear_model import LogisticRegression
v=VotingClassifier([('lr',LogisticRegression(max_iter=300)),('rf',RandomForestClassifier())],voting='soft')
```

Learn more
YouTube search: Stacking and Voting ensembles tutorial

13. Support Vector Machines (SVM) and kernels [Intermediate]

Explanation (English)
Finds the maximum-margin hyperplane; kernel trick (RBF, polynomial) handles non-linear boundaries; C controls margin vs errors. Works well on small/medium data; scale features.

Tanglish (Tamil + English) Vilakkam
SVM max margin line/hyperplane. Kernel trick non-linear data-ku. C, gamma tune.

Example / Program
```
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
X,y=load_iris(return_X_y=True)
Xtr,Xte,ytr,yte=train_test_split(X,y,random_state=0)
print(SVC(kernel='rbf',C=1).fit(Xtr,ytr).score(Xte,yte))
```

Learn more
YouTube search: Support Vector Machines (SVM) and kernels tutorial

14. Model evaluation: metrics for classification [Beginner]

Explanation (English)
Confusion matrix, accuracy, precision, recall, F1, ROC-AUC, PR-AUC, log loss. Choose by cost of errors (e.g. recall for disease detection).

Tanglish (Tamil + English) Vilakkam
Accuracy mattum podhaadhu. Precision, recall, F1, ROC-AUC paarkanum.

Example / Program
```
from sklearn.metrics import classification_report,confusion_matrix
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
X,y=load_iris(return_X_y=True)
Xtr,Xte,ytr,yte=train_test_split(X,y,random_state=0)
p=LogisticRegression(max_iter=300).fit(Xtr,ytr).predict(Xte);print(confusion_matrix(yte,p),classification_report(yte,p))
```

Learn more
YouTube search: Model evaluation: metrics for classification tutorial

15. Model evaluation: metrics for regression [Beginner]

Explanation (English)
MAE, MSE, RMSE, R2, adjusted R2, MAPE. RMSE penalises large errors; MAE is robust.

Tanglish (Tamil + English) Vilakkam
MAE, MSE, RMSE, R2 - regression metrics.

Example / Program
```
from sklearn.metrics import mean_squared_error,r2_score
print(mean_squared_error([3,5],[2.5,5.5]),r2_score([3,5,7],[2.5,5.5,6.5]))
```

Learn more
YouTube search: Model evaluation: metrics for regression tutorial

16. Cross-validation, grid/random search, hyper-parameter tuning [Intermediate]

Explanation (English)

K-fold, stratified K-fold, time-series split give robust estimates. GridSearchCV/RandomizedSearchCV/Optuna search hyper-parameters. Keep a final untouched test set.

Tanglish (Tamil + English) Vilakkam

K-fold cross validation, GridSearchCV. Final test set-a thottaadheenga.

Example / Program

```
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
X,y=load_iris(return_X_y=True)
Xtr,Xte,ytr,yte=train_test_split(X,y,random_state=0)
g=GridSearchCV(SVC(),{'C':[.1,1,10]},cv=5).fit(Xtr,ytr);print(g.best_params_,g.best_score_)
```

Learn more

YouTube search: Cross-validation, grid/random search, hyper-parameter tuning tutorial

17. Probability calibration and threshold selection [Advanced]

Explanation (English)

Make predicted probabilities reliable (Platt scaling, isotonic); choose thresholds using business costs and PR/ROC curves.

Tanglish (Tamil + English) Vilakkam

Probability correct-a irukkanum - calibration. Threshold business cost paathu choose pannunga.

Example / Program

```
from sklearn.calibration import CalibratedClassifierCV
from sklearn.svm import LinearSVC
m=CalibratedClassifierCV(LinearSVC())
```

Learn more

YouTube search: Probability calibration and threshold selection tutorial

18. Clustering: K-Means, K-Means++, elbow and silhouette [Beginner]

Explanation (English)

Partition unlabeled data into k clusters minimising within-cluster variance. Choose k with elbow/silhouette. Sensitive to scale and initialisation.

Tanglish (Tamil + English) Vilakkam

K-Means data-va k groups-a pirikkum. Elbow, silhouette-la k select.

Example / Program

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.datasets import make_blobs
X,_=make_blobs(300,centers=3,random_state=0)
k=KMeans(3,n_init=10).fit(X);print(silhouette_score(X,k.labels_))
```

Learn more

YouTube search: Clustering: K-Means, K-Means++, elbow and silhouette tutorial

19. Clustering: Hierarchical, DBSCAN, Gaussian Mixture, Spectral [Intermediate]

Explanation (English)

Hierarchical builds a dendrogram; DBSCAN finds arbitrary-shaped clusters and noise; GMM gives soft probabilistic clusters.

Tanglish (Tamil + English) Vilakkam

Hierarchical dendrogram, DBSCAN noise detect, GMM soft clusters.

Example / Program

```
from sklearn.cluster import DBSCAN,AgglomerativeClustering
from sklearn.mixture import GaussianMixture
from sklearn.datasets import make_moons
X,_=make_moons(200,noise=.05,random_state=0)
print(set(DBSCAN(eps=.2).fit_predict(X)))
```

Learn more

YouTube search: Clustering: Hierarchical, DBSCAN, Gaussian Mixture, Spectral tutorial

20. Anomaly / outlier detection [Intermediate]

Explanation (English)

Isolation Forest, One-Class SVM, Local Outlier Factor, autoencoders, z-score. Used in fraud and fault detection.

Tanglish (Tamil + English) Vilakkam

Fraud, fault detect panna - Isolation Forest, LOF.

Example / Program

```
from sklearn.ensemble import IsolationForest
import numpy as np
X=np.r_[np.random.randn(100,2),[[8,8]]]
print(IsolationForest(random_state=0).fit_predict(X)[-1])
```

Learn more

YouTube search: Anomaly / outlier detection tutorial

21. Association rules: Apriori / FP-Growth (market basket) [Intermediate]

Explanation (English)

Find items frequently bought together using support, confidence and lift.

Tanglish (Tamil + English) Vilakkam

Market basket analysis: support, confidence, lift.

Example / Program

```
from mlxtend.frequent_patterns import apriori,association_rules
# apriori(df,min_support=.2,use_colnames=True)
```

Learn more

YouTube search: Association rules: Apriori / FP-Growth (market basket) tutorial

22. Recommender systems: content-based and collaborative filtering [Intermediate]

Explanation (English)

Content-based uses item features; collaborative filtering uses user-item interactions (user/item KNN, matrix factorisation/SVD, implicit feedback, two-tower neural models).

Tanglish (Tamil + English) Vilakkam

Recommendation: content-based, collaborative filtering, matrix factorization (Netflix, YouTube).

Example / Program

```
import numpy as np
R=np.array([[5,3,0],[4,0,1],[1,1,5]],float)
U,S,Vt=np.linalg.svd(R);print((U[:,:2]*S[:2])@Vt[:2])
```

Learn more

YouTube search: Recommender systems: content-based and collaborative filtering tutorial

23. Semi-supervised, self-supervised and active learning [Advanced]

Explanation (English)

Use few labels plus many unlabelled samples (pseudo-labelling, label propagation), learn representations by predicting parts of the data (contrastive/masked), or query the most informative samples to label.

Tanglish (Tamil + English) Vilakkam

Konjam label + niraya unlabeled data. Self-supervised = data-va vachee thaane label create.

Example / Program

```
from sklearn.semi_supervised import LabelPropagation
```

Learn more

YouTube search: Semi-supervised, self-supervised and active learning tutorial

24. Time-series forecasting (classical): moving average, ARIMA, SARIMA, Prophet [Intermediate]

Explanation (English)

Decompose trend/seasonality/noise; stationarity (ADF test); ARIMA(p,d,q); Prophet for business forecasts; evaluate with rolling-origin validation.

Tanglish (Tamil + English) Vilakkam

Trend, seasonality, ARIMA, Prophet. Time order maaraama split pannunga.

Example / Program

```
from statsmodels.tsa.arima.model import ARIMA
import numpy as np
y=np.cumsum(np.random.randn(100))
print(ARIMA(y,order=(1,1,1)).fit().forecast(5))
```

Learn more

YouTube search: Time-series forecasting (classical): moving average, ARIMA, SARIMA, Prophet tutorial

25. Interpretability: feature importance, permutation importance, PDP, SHAP, LIME [Advanced]

Explanation (English)

Explain models globally and locally; permutation importance is model-agnostic; SHAP gives consistent attributions.

Tanglish (Tamil + English) Vilakkam

Model eppadi decide pannudhu-nu explain: permutation importance, SHAP, LIME.

Example / Program

```
from sklearn.inspection import permutation_importance
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
X,y=load_iris(return_X_y=True)
Xtr,Xte,ytr,yte=train_test_split(X,y,random_state=0)
m=RandomForestClassifier().fit(Xtr,ytr);print(permutation_importance(m,Xte,yte,random_state=0).importances_mean)
```

Learn more

YouTube search: Interpretability: feature importance, permutation importance, PDP, SHAP, LIME tutorial

26. Saving models and reproducibility [Beginner]

Explanation (English)

Persist with joblib/pickle/ONNX; fix random seeds; record library versions; use pipelines and config files.

Tanglish (Tamil + English) Vilakkam

joblib-la model save. random_state fix pannunga.

Example / Program

```
import joblib
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
X,y=load_iris(return_X_y=True)
Xtr,Xte,ytr,yte=train_test_split(X,y,random_state=0)
m=LogisticRegression(max_iter=300).fit(Xtr,ytr);joblib.dump(m,'m.joblib');print(joblib.load('m.joblib').score(Xte,yte))
```

Learn more

YouTube search: Saving models and reproducibility tutorial

27. End-to-end ML project template [Intermediate]

Explanation (English)

Business question -> data -> EDA -> baseline -> pipeline -> CV -> tune -> error analysis -> explain -> save -> API -> monitor. Always beat a simple baseline first.

Tanglish (Tamil + English) Vilakkam

Business question-la start panni, baseline model, pipeline, CV, tune, error analysis, deploy.

Learn more

YouTube search: end to end machine learning project scikit-learn